
Informe de Ambientes

Proyecto: Autogestión-Sena

ÍNDICE

1. Introducción
2. Objetivos del documento
3. Estructura de Ambientes en GitHub
4. Flujo de trabajo (Workflow) por ambientes
5. Justificación del manejo de ambientes
6. Políticas internas de ramas (Branching Policy)
7. Control de versiones y convenciones de commits
8. Uso de ambientes por repositorio
9. Problemas encontrados y soluciones implementadas
10. Ejemplos con capturas (Frontend)
11. Repositorios
12. Riesgos identificados del flujo de ambientes
13. Uso de Cherry Pick y Fork
14. Impacto del manejo de ambientes en la calidad del proyecto
15. Conclusiones

1. Introducción

El presente documento tiene como propósito describir de forma amplia, detallada y estructurada el proceso de manejo de ambientes utilizado en el proyecto Autogestión SENA, así como las herramientas, flujos de trabajo, estrategias y dificultades encontradas durante su implementación.

En proyectos de desarrollo de software modernos, la gestión adecuada del control de versiones es un factor crítico para garantizar la calidad, estabilidad y trazabilidad del código. Por esta razón, el equipo adoptó una arquitectura de ramas basada en un modelo similar a GitFlow, aplicando ambientes independientes para cada etapa del ciclo de desarrollo:

- dev → ambiente de desarrollo activo.
- qa → ambiente de pruebas funcionales
- staging → ambiente de preproducción.
- main → ambiente estable o de producción.

Este esquema proporciona un marco claro para la evolución del software, permitiendo que las funcionalidades (HU – Historias de Usuario) avancen por fases de validación progresiva antes de ser integradas en el producto final.

Cada HU cuenta con una rama por ambiente, lo que facilita aislar cambios, revisar avances y detectar errores sin afectar el proyecto global. De esta manera, se garantiza que los desarrollos cumplan con estándares de calidad y se integren únicamente cuando estén aprobados.

Sin embargo, este proceso no estuvo exento de dificultades. Aunque el manejo de ambientes se implementó con éxito en el frontend (React + TypeScript), los repositorios de backend (Django REST Framework) y móvil (.NET MAUI) presentaron limitaciones derivadas de factores como diferencias en entornos locales, conflictos de migraciones, cambios rápidos en UI y problemas de sincronización. Aun así, se documentó todo el proceso como evidencia formativa y como parte del aprendizaje obtenido durante la construcción del sistema.

Este documento expone:

- Cómo se estructuraron las ramas.
- Qué propósito tiene cada ambiente
- Cómo se gestionaron las HU en cada etapa.
- Qué herramientas se usaron (como cherry-pick y Fork).
- Qué problemas surgieron y cómo se resolvieron.
- Evidencias con capturas y ejemplos reales (HU-42 y HU-43).

Su propósito final es dejar un registro académico completo del proceso de control de versiones aplicado, documentando tanto los logros como las dificultades para que pueda servir como referencia para proyectos futuros y como evidencia del trabajo realizado por el equipo.

2. Objetivos del documento

- Objetivo general:
 - Describir de manera completa el proceso de manejo de ambientes realizado en GitHub para el proyecto Autogestión SENA.
- Objetivos específicos:
 - Explicar la función de cada ambiente (dev, qa, staging, main).
 - Documentar el flujo de trabajo por cada Historia de Usuario.
 - Registrar el uso de ramas temporales por HU.
 - Detallar el proceso de integración (MR) entre ambientes.
 - Evidenciar el uso de cherry pick para resolver conflictos o trasladar cambios específicos.
 - Presentar ejemplos reales ejecutados durante el proyecto (HU-42 y HU-43).
 - Dejar registro del funcionamiento exitoso en frontend y de las dificultades en backend y móvil.

3. Estructura de Ambientes en GitHub

3.1. ¿Qué es un ambiente en control de versiones?

Un ambiente es un espacio lógico que representa un **estado del software** dentro del ciclo de vida.

Cada ambiente permite evaluar el proyecto en diferentes etapas:

- **Desarrollo**
- **Pruebas**
- **Preproducción**
- **Producción**

Esto evita mezclar código experimental con código estable y garantiza calidad en los despliegues.

3.2. Ambientes utilizados en Autogestión SENA

El proyecto utiliza un modelo basado en **GitFlow**, en el que cada ambiente representa una fase de maduración del software:

Ambiente	Propósito
dev	Desarrollo diario, nuevas funcionalidades y correcciones.
qa	Pruebas funcionales y validaciones internas.
staging	Simulación de producción antes del lanzamiento.
main	Versión estable y lista para despliegue final.

Cada Historia de Usuario (HU) crea **una rama por ambiente**, con el patrón:

- HU-XX-dev
- HU-XX-qa
- HU-XX-staging

Solo en el ambiente **main** se utiliza una rama especial:

- release.X.X

4. Flujo de trabajo (Workflow) por ambientes

A continuación se documenta el flujo exacto utilizado para cada ambiente, basado en ramas por HU (Historia de Usuario).

4.1. Ambiente *dev*

Es donde inicia el desarrollo de cada HU.

Es un ambiente dinámico y cambiante.

Procedimiento:

```
git pull origin dev
git switch dev
git switch -c HU-01-dev
# → desarrollo
# → commit y push
Crear MR desde: HU-01-dev → dev
```

Se cierra el desarrollo cuando la rama **dev** queda actualizada con la HU aprobada.

Acciones:

- Desarrollo de la funcionalidad.
- Corrección de errores iniciales.
- Integración con otras HU.

Se finaliza con un **merge request (MR)** hacia *dev*.

4.2. Ambiente *qa*

Para pruebas funcionales y validaciones.

Procedimiento:

```
git pull origin qa  
git switch qa  
git switch -c HU-01-qa  
# → desarrollo QA  
Crear MR: HU-01-qa → qa
```

Se cierra una vez la HU se aprueba en las pruebas.

Acciones:

- Verificación funcional.
- Validación de requisitos.
- Revisión por parte del equipo.

MR hacia *qa*.

4.3. Ambiente *staging*

Antes de llevar a main, staging permite probar el sistema completo.

Procedimiento:

```
git pull origin staging  
git switch staging  
git switch -c HU-01-staging  
# → ajustes finales  
Crear MR: HU-01-staging → staging
```

Esta rama simula el entorno real antes de producción.

Acciones:

- Pruebas finales.
- Validación previa al release.
- Revisión del comportamiento real del sistema.

MR hacia staging.

4.4. Ambiente main

Es la rama más importante: código listo para publicación.

Procedimiento:

```
git pull origin main  
git switch main  
git switch -c release.1.1
```

Dentro de esa release ingresan todas las HU del sprint:

```
HU-01-release.1.1  
HU-02-release.1.1  
HU-03-release.1.1  
...
```

Luego:

Crear MR: release.1.1 → main

Nunca se debe trabajar directamente sobre main.

Flujo de trabajo por HU:

1. Crear rama en **dev**
2. Desarrollar HU
3. Merge HU → dev
4. Crear rama HU en **qa**
5. Pruebas
6. Merge HU → qa
7. Crear rama HU en **staging**
8. Pruebas pre-release
9. Merge HU → staging
10. Crear rama release
11. Merge release → main

5. Justificación del manejo de ambientes

El uso de múltiples ambientes en GitHub se justifica por varias razones técnicas y metodológicas que buscan garantizar la estabilidad del proyecto:

- Seguridad del código

Trabajar mediante entornos separados reduce riesgos de que errores en desarrollo afecten al código estable o al que está listo para despliegue.

- Control granular del ciclo de vida

Cada Historia de Usuario atraviesa un proceso estructurado que permite identificar fallos, validar funcionalidad y mantener coherencia entre módulos.

- Facilita el trabajo en equipo

Evita que los desarrolladores trabajen sobre la misma rama principal, reduciendo conflictos, duplicación de cambios y pérdida de información.

- Es compatible con metodologías ágiles

Scrum exige entregas frecuentes y controladas; GitFlow permite estructurar estas entregas mediante releases.

6. Políticas internas de ramas (Branching Policy)

Para estandarizar el manejo de ambientes se definieron lineamientos internos:

- Reglas de nombramiento

HU-XX-dev

HU-XX-qa

HU-XX-staging

release.X.X

- Reglas de merge

- ❖ No se permite trabajar directo sobre dev, qa, staging o main.
- ❖ Los merges a ramas principales solo se realizan desde HU o release.
- ❖ Todo merge debe ser revisado por al menos un integrante del equipo.

- Reglas de commits

- ❖ Commits cortos y descriptivos.
- ❖ Prohibido mezclar cambios de varias HU en una sola rama

7. Control de versiones y convenciones de commits

Durante el desarrollo del proyecto se aplicó un esquema estandarizado para el manejo de versiones utilizando Git y GitHub. Con el fin de mantener un historial consistente, ordenado y fácil de auditar, el equipo adoptó una convención uniforme para los mensajes de commit, escrita exclusivamente en inglés, siguiendo una estructura clara que permite identificar rápidamente el tipo de cambio realizado.

Estructura utilizada para los commits

Cada commit se redactó siguiendo el formato:

<type>: <short description>

Files: <modified files>

Detail: <what was done>

Tipos de commit utilizados

Se emplearon etiquetas breves que describen el propósito del cambio:

- feat: para nuevas funcionalidades.
- fix: para corrección de errores.
- update / refactor: para mejoras internas, limpieza o reestructuración del código.
- style: para ajustes de formato o UI.
- docs: para modificaciones en la documentación.
- chore: tareas menores del proyecto.

8. Uso de ambientes por repositorio

8.1. Frontend Web (React + TS + Vite)

Este repositorio fue el que sí logró implementar exitosamente todo el manejo de ambientes.

- Se trabajaron ramas dev, qa, staging y main.
- Se crearon ramas por HU por cada ambiente.
- Se hicieron merges controlados.
- Mantiene commits limpios y un flujo ordenado.

8.2. Backend (Django REST Framework)

En el backend se intentó implementar el mismo flujo, pero no se logró completamente.

Razones:

- Dependencias rotas entre ramas.
- Migraciones de base de datos generaban conflictos.
- Diferencias entre entornos locales impedían mover código fácilmente.
- Se trabajó más directo sobre dev y main.

Resultado: el manejo de ambientes no fue tan sólido como en frontend.

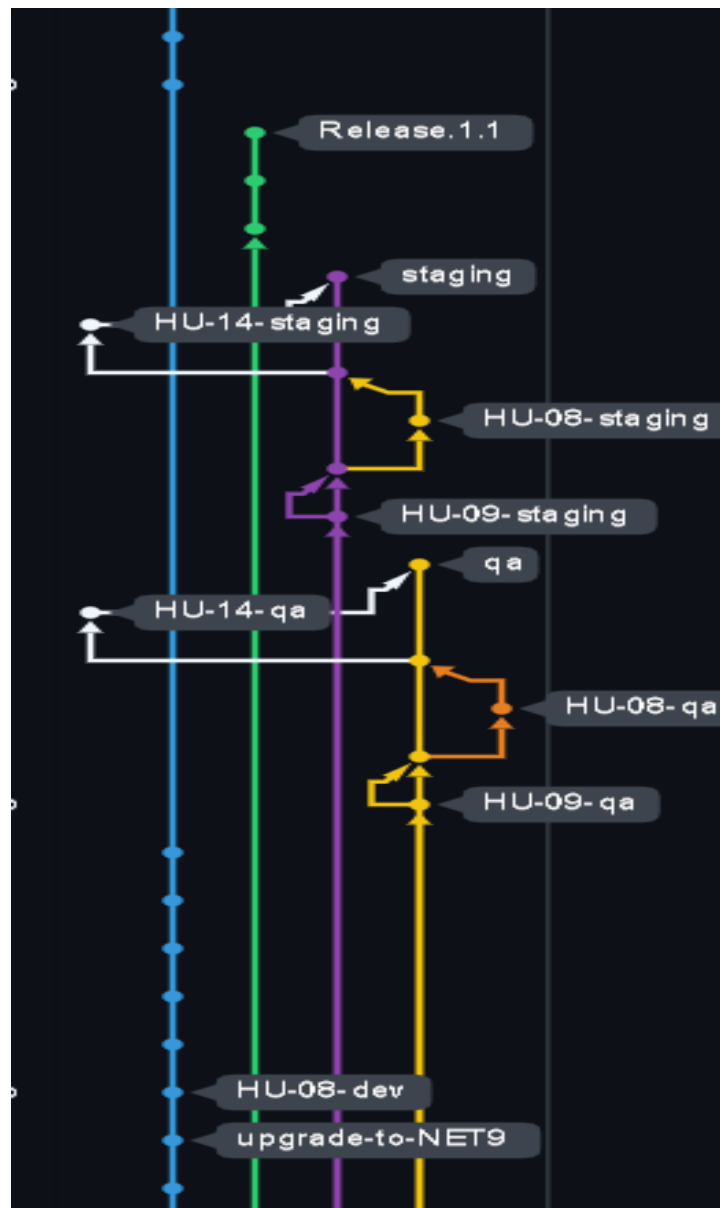
8.3. Aplicación Móvil (.NET MAUI)

También se intentó el manejo de ambientes, pero surgieron fallos.

- La UI cambiaba constantemente.
- Problemas de sincronización entre repositorios.
- Se avanzó más lento debido al enfoque visual.

El flujo se implementó parcialmente.

Se añade una imagen de ejemplo del inicio del flujo:



9. Problemas encontrados y soluciones implementadas

Problema	Causa	Solución
Conflictos entre ramas	Migraciones y cambios simultáneos	Establecer un orden de commits y revisar antes de merge
Fallo en control de versiones del móvil	Cambios rápidos	Acordar estándares de commits
CI fallaba por dependencias	Versiones diferentes	Unificar versiones en equipo
Muchas ramas abiertas	Falta de cierre	Políticas de merge estrictas

10. Ejemplos con capturas (Frontend)

Aclaracion:

Durante el desarrollo se implementó el uso de cherry-pick para trasladar cambios específicos entre ramas y asegurar la correcta gestión de cada Historia de Usuario (HU), apoyándonos en la herramienta Fork para facilitar este proceso.

Debido a dificultades iniciales con el manejo de ambientes en los diferentes repositorios, se decidió consolidar un ambiente final en el frontend, aplicando correctamente el flujo dev → qa → staging → main. En este ambiente se integraron y documentaron los cambios finales y las HU aprobadas, con el objetivo de demostrar y evidenciar el proceso de gestión de ambientes que se aprendió e implementó durante el proyecto.

Ejemplo real HU-42

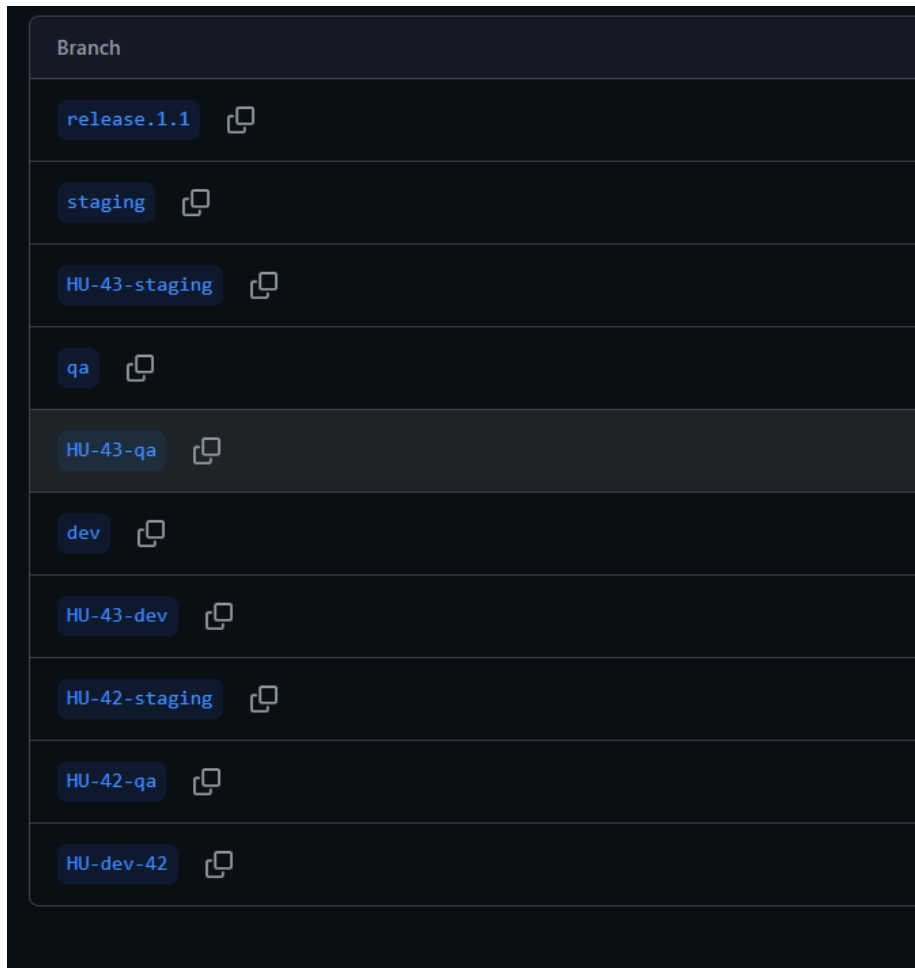
Para **HU-42** se siguió este proceso:

1. Se creó **HU-42-dev**
2. Se desarrolló la funcionalidad
3. Se hizo merge hacia **dev**
4. Se creó **HU-42-qa** y se pasó con **cherry pick**
5. Luego se pasó a **HU-42-staging**
6. Se integró con staging
7. Se incluyó en el release final

Ejemplo real HU-43

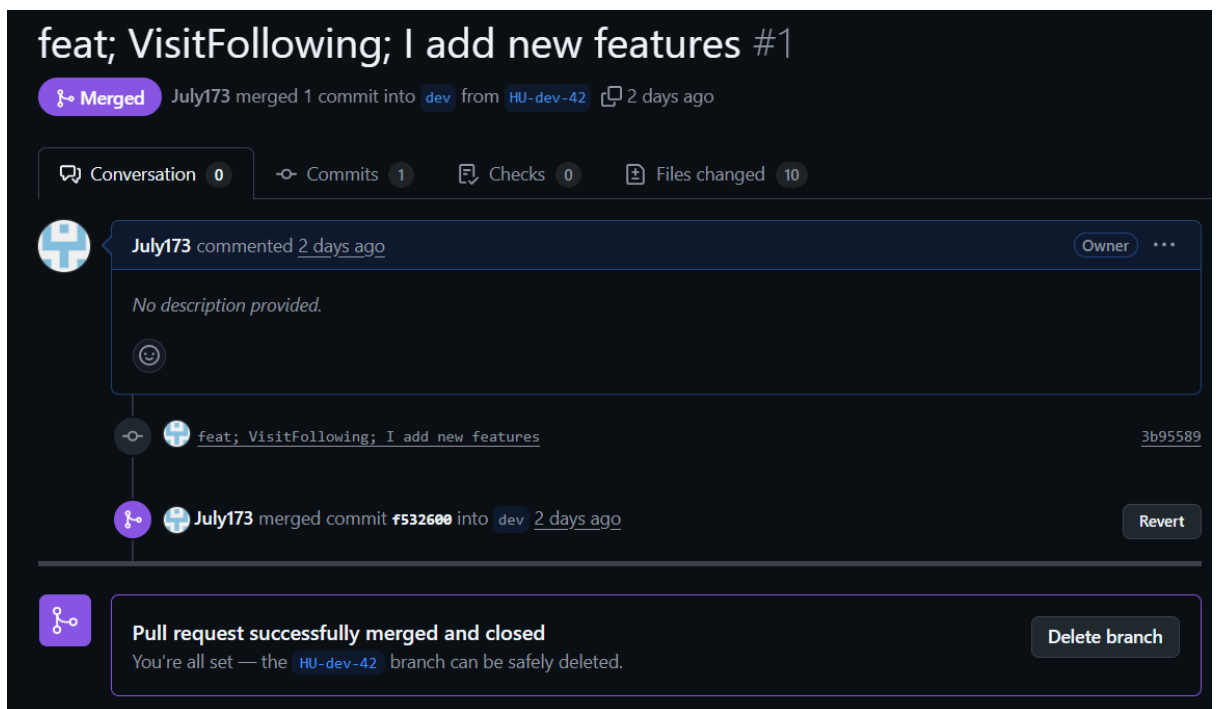
La **HU-43** repitió el proceso:

- HU-43-dev → HU-43-qa → HU-43-staging
- Se aplicaron merge request entre ambientes
- Finalmente pasó al release
- Y del release se integró a main

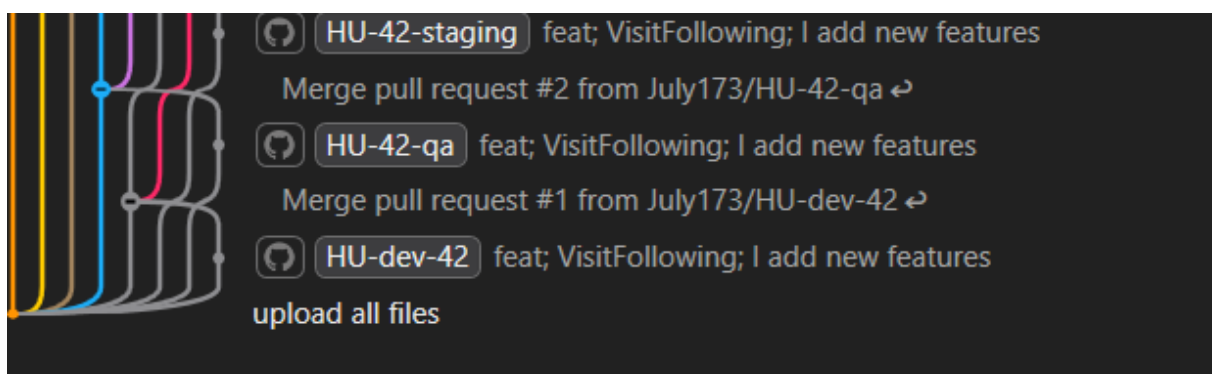


dev:

Se creo la HU-42-dev se hizo el proceso y se hizo el merge a la dev:



Posteriormente, los cambios desarrollados en la rama **HU-42-dev** fueron promovidos hacia la rama **HU-42-qa**, y luego hacia **HU-42-staging**, siguiendo el flujo de ambientes establecido. Durante este proceso se realizaron los **merge correspondientes entre cada HU y su rama principal**, es decir, primero desde la HU hacia **qa**, y luego desde la HU hacia **staging**, garantizando que cada nivel recibiera únicamente los cambios aprobados y validados para ese ambiente.



feat; VisitFollowing; I add new features #2

Merged July173 merged 1 commit into `qa` from `HU-42-qa` 2 days ago

Conversation 0 Commits 1 Checks 0 Files changed 10

July173 commented 2 days ago Owner ...

No description provided.

feat; VisitFollowing; I add new features 20d3d95

July173 merged commit `4bca483` into `qa` 2 days ago Revert

Pull request successfully merged and closed Delete branch

You're all set — the `HU-42-qa` branch can be safely deleted.

feat; VisitFollowing; I add new features #3

Merged July173 merged 1 commit into `staging` from `HU-42-staging` 2 days ago

Conversation 0 Commits 1 Checks 0 Files changed 10

July173 commented 2 days ago Owner ...

No description provided.

feat; VisitFollowing; I add new features a9cd0b4

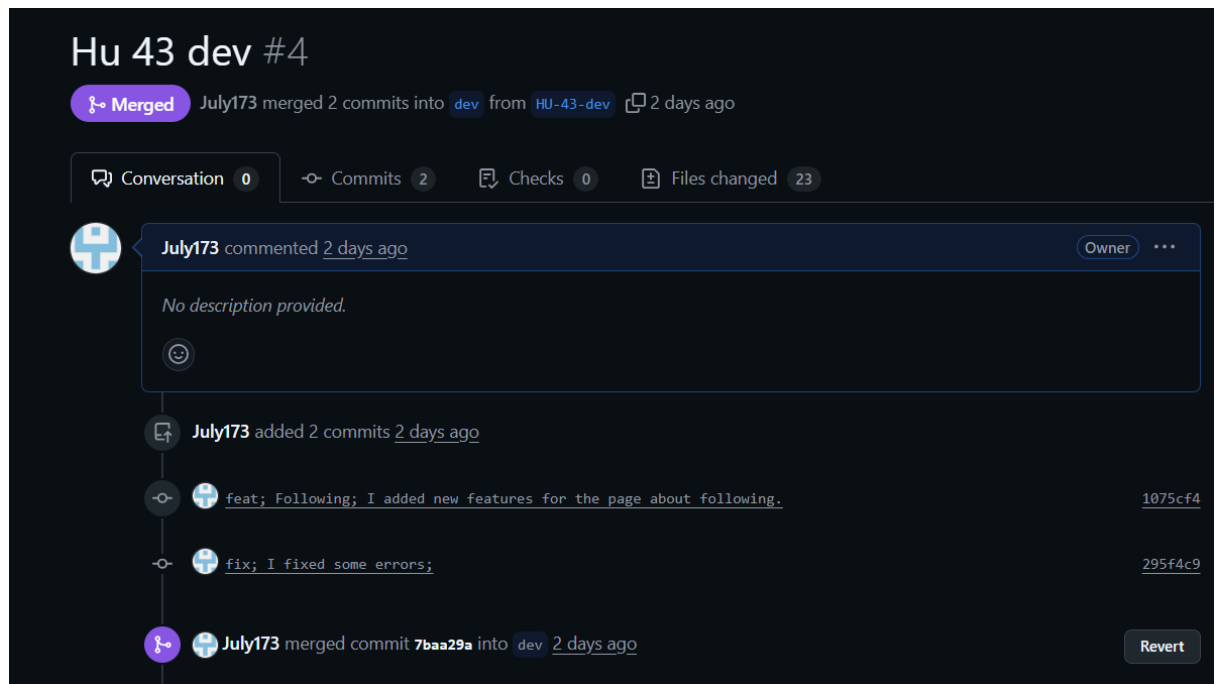
July173 merged commit `bfca153` into `staging` 2 days ago Revert

Pull request successfully merged and closed Delete branch

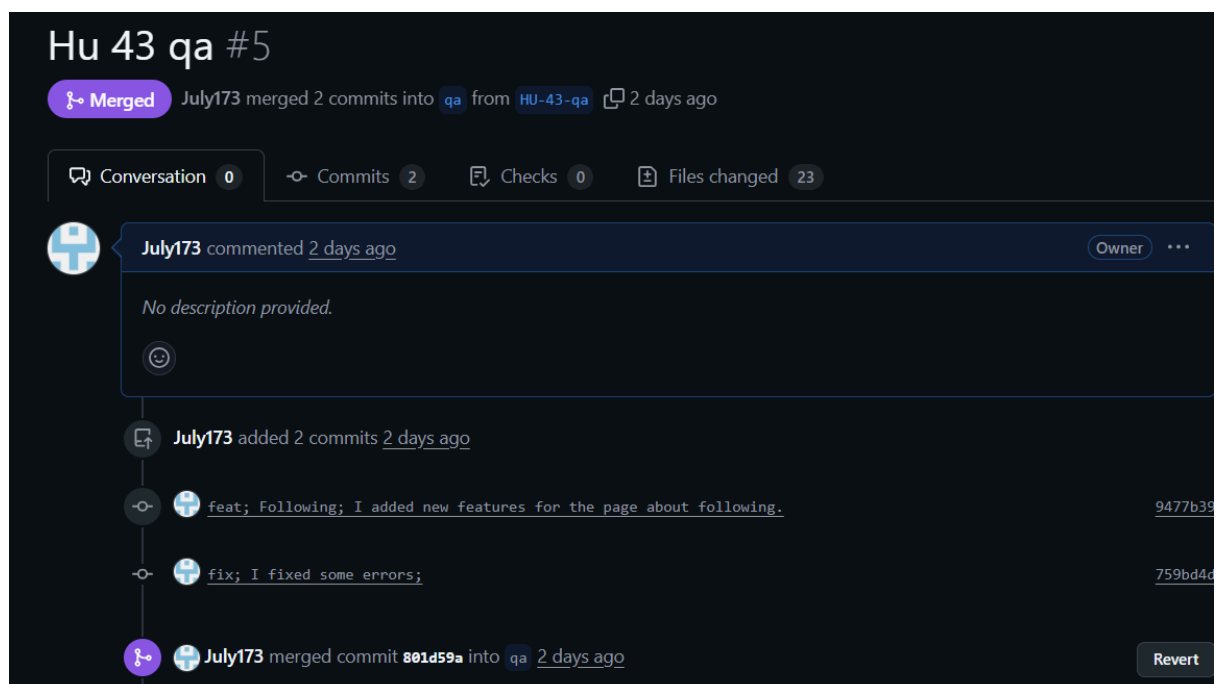
You're all set — the `HU-42-staging` branch can be safely deleted.

Posteriormente, se aplicó el mismo flujo de trabajo con la HU-43, trasladando los cambios desde la rama **HU-43-dev** hacia **HU-43-staging**, siguiendo las etapas correspondientes.

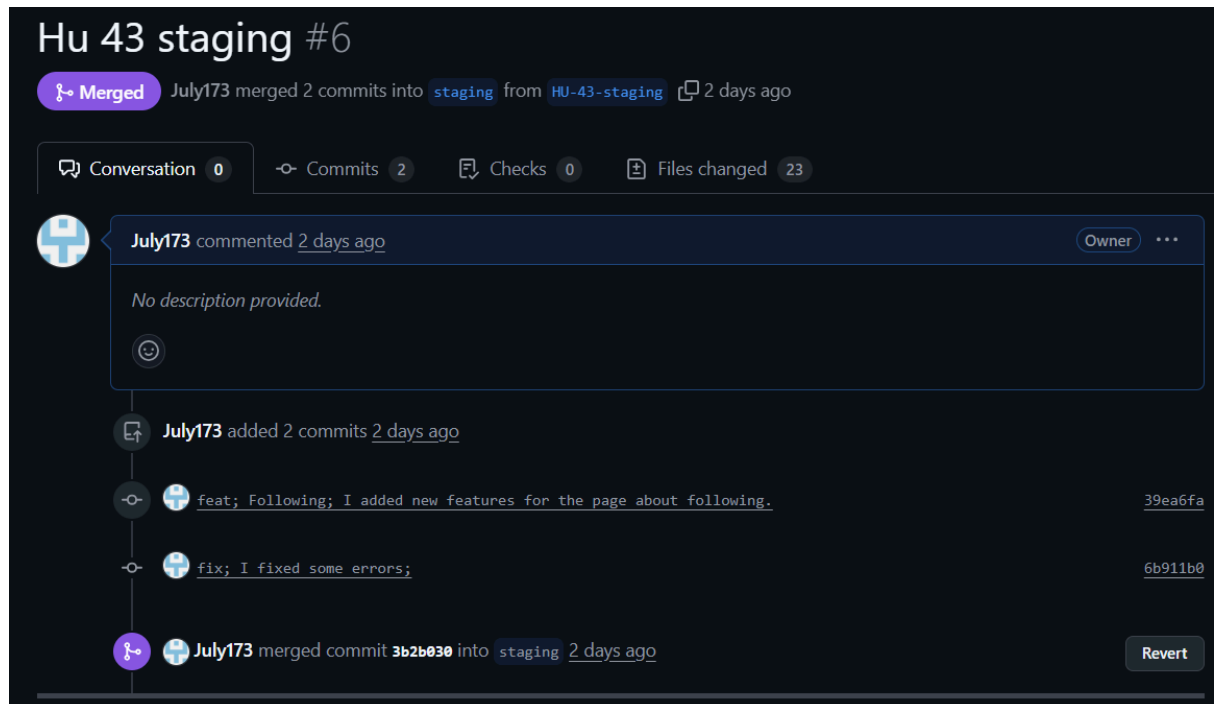
En la siguiente imagen se muestra que se hizo el merge de la HU-43-dev a la dev:



Luego se creo la rama HU-43-qa a partir de la qa y se hicieron los cherry pick de la rama HU-43-dev a la HU-43-qa se hizo el merge de la HU-43-qa a la qa:

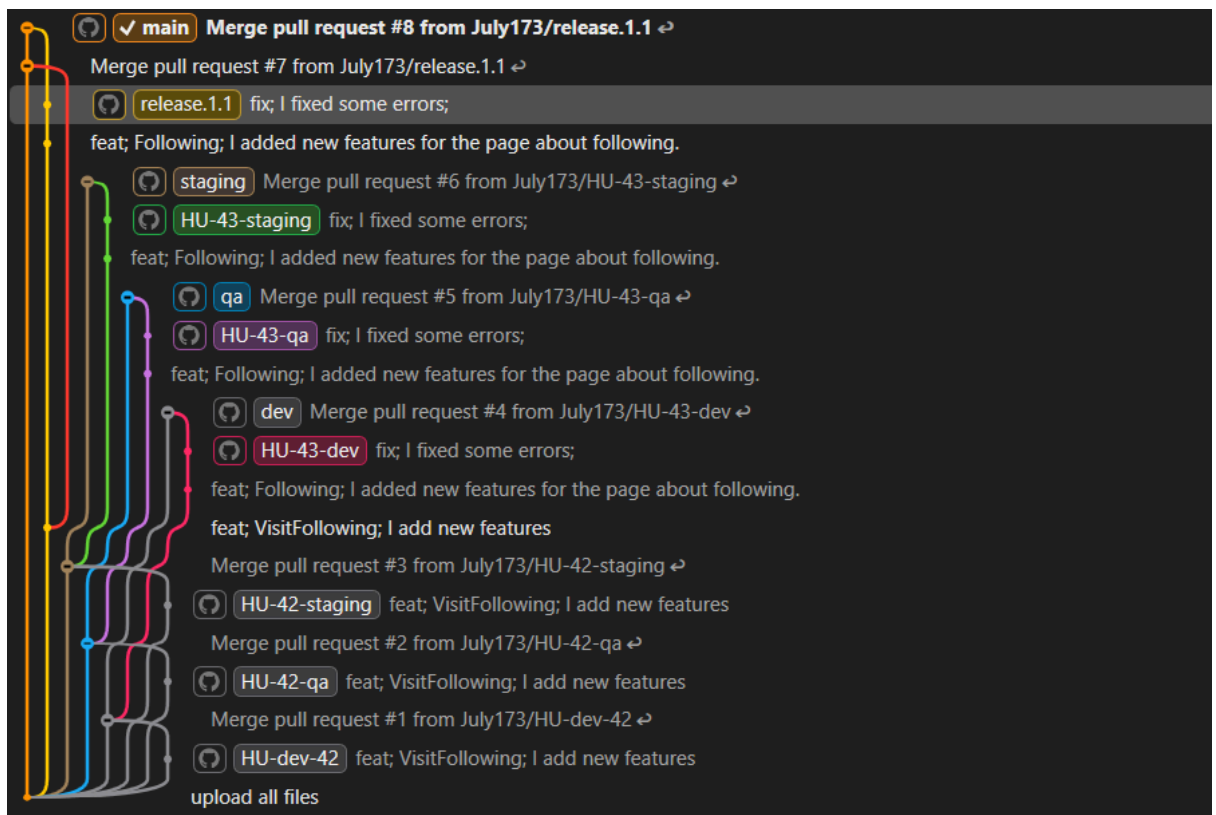


Luego se creo la rama HU-43-staging a partir de la staging y se hicieron los cherry pick de la rama HU-43-qa a la HU-43-staging se hizo el merge de la HU-43-staging a la staging:



Una vez validadas las historias de usuario en los entornos previos, se procedió a generar los **release**, consolidando las funcionalidades aprobadas. Pasando de las HU-staging con cherry pick a la release 1.1. Finalmente, estos cambios fueron integrados en la rama **main**, completando el ciclo de publicación.

Este es el proceso total visualizado con la herramienta fork:



Release.1.1 #8

Merged July173 merged 2 commits into `main` from `release.1.1` 1 hour ago

Conversation 0 Commits 2 Checks 0 Files changed 23

July173 commented 1 hour ago

No description provided.

July173 added 2 commits yesterday

- feat; Following; I added new features for the page about following. [f11d3bc](#)
- fix; I fixed some errors; [f57ce65](#)

July173 merged commit [631dedb](#) into `main` 1 hour ago [Revert](#)

11. Repositorios GitHub:

- **Frontend:**

- <https://github.com/July173/autogestionFrontWeb>

- **Backend:**

- <https://github.com/July173/backAuotesgestion>

- **Móvil:**

- <https://github.com/July173/FrontMovilAutogestion>

12. Riesgos identificados del flujo de ambientes

Durante la implementación del flujo de ambientes en el proyecto Autogestión SENA, se identificaron diversos riesgos asociados tanto al uso de GitHub como a la gestión del código en múltiples ramas (dev, qa, staging y main). Estos riesgos afectan la estabilidad del producto, la coherencia del código y la capacidad del equipo para integrar funcionalidades de manera ágil. A continuación se describen los principales riesgos detectados, su impacto potencial y las causas técnicas que los generan.

1. Riesgo de conflictos entre ramas al integrar HU

El manejo de múltiples ramas por Historia de Usuario incrementa la probabilidad de conflictos. Cuando dos desarrolladores trabajan sobre módulos relacionados, las modificaciones se superponen, especialmente en archivos centrales como rutas, controladores o componentes globales.

Impacto:

- Retrasos en el flujo dev → qa → staging.
- Conflictos complejos que requieren refactorización.
- Posible pérdida de cambios si no se gestionan adecuadamente.

Causa:

Ediciones simultáneas sobre archivos críticos, estructuras compartidas o dependencias.

2. Riesgo por diferencias en entornos locales

Cada miembro del equipo contaba con configuraciones distintas en su entorno local (versiones de Node, Python, librerías, dependencias), lo que generó incompatibilidades al momento de ejecutar el proyecto o generar builds.

Impacto:

- Fallos inesperados al pasar de un ambiente a otro.
- Inestabilidad en migraciones en backend.

Causa:

Falta de estandarización de versiones y uso limitado de herramientas de automatización.

3. Riesgo por no completar correctamente el flujo GitFlow

Aunque en frontend sí se logró implementar completamente la estructura de ambientes, en backend y móvil aparecieron dificultades que impidieron mantener un flujo homogéneo.

Impacto:

- Algunas HU no pudieron seguir el camino dev → qa → staging.
- Falta de trazabilidad total de cambios en ciertos repositorios.

Causa:

Problemas con migraciones, dependencias y diferencias de sincronización.

4. Riesgo de cherry-pick incorrecto

El cherry-pick fue fundamental, pero su uso incorrecto puede introducir commits duplicados, inconsistencias o errores de ejecución.

Impacto:

- Historial con commits repetidos o en orden incorrecto.
- Conflictos inesperados en staging o main.

Causa:

Errores al seleccionar commits o falta de revisión previa.

5. Riesgo de merges prematuros

Hacer merge antes de que una HU sea verificada completamente afecta la calidad del ambiente.

Impacto:

- Bugs que llegan a qa o staging sin ser detectados.
- Retrocesos y reversiones costosas.

Causa:

Falta de revisión en Pull/Merge Requests o presión por tiempos.

6. Riesgo por demasiadas ramas abiertas

Durante el desarrollo, existieron momentos donde varias HU permanecían abiertas y sin cierre.

Impacto:

- Mayor complejidad para sincronizar cambios.
- Aumento de conflictos de fusión.

Causa:

Falta de políticas estrictas de cierre y mantenimiento del repositorio.

7. Riesgo en Staging como ambiente crítico

Staging funciona como preproducción. Cualquier error que llegue allí desde dev o qa afecta directamente al release.

Impacto:

- Retrasos en entregas finales.
- Necesidad de rehacer varias etapas del flujo.

Causa:

Fallas al validar correctamente las HU antes del paso a staging.

8. Riesgo de desactualización entre ambientes

En ocasiones, dev avanzaba más rápido que qa o staging, causando que estos ambientes quedaran desalineados con los cambios reales.

Impacto:

- Inconsistencias al momento de crear release.
- Pérdida de control sobre el estado real del proyecto.

Causa:

Sincronización manual deficiente y acumulación de HU pendientes.

13. Uso de Cherry Pick y Fork

En este proyecto, los **cherry-pick** se utilizaron para trasladar commits específicos de una HU a otra HU cuando era necesario compartir cambios o corregir problemas entre ramas que **no podían fusionarse por merge directo**.

Esto se debe a que:

- Las HU son ramas independientes que no deben mezclarse directamente.
- No se pueden hacer merges entre ramas HU-xx-dev o entre HU-xx-qa.
- Para mover un commit desde una HU a otra se debe hacer:

```
git cherry-pick <id_commit>
```

Los cherry-pick se realizaron en múltiples casos, sobre todo durante:

- El proceso de HU-42 (dev → qa → staging).
- El proceso posterior de HU-43.
- La integración final en los releases.

14. Impacto del manejo de ambientes en la calidad del proyecto

La implementación de una estrategia formal de manejo de ambientes (dev → qa → staging → main) tuvo un impacto significativo en la calidad, estabilidad y organización del proyecto Autogestión SENA. Aunque la metodología no se logró aplicar al 100 % en backend y móvil por limitaciones técnicas, su implementación completa en el frontend permitió evidenciar claramente los beneficios del enfoque.

1. Mejora en la calidad del código

El uso de ambientes separados permitió que cada funcionalidad se desarrollara, probara y validara de manera escalonada. Gracias a esto:

El código que llegaba a main ya había pasado por varias capas de revisión.

Se redujo la cantidad de errores críticos al momento de hacer el release.

Se evitaron mezclas de código experimental con código estable, algo común en proyectos sin ambientes.

Esto garantizó un producto final más limpio y depurado.

2. Reducción de errores en producción

Al validar cada Historia de Usuario primero en dev, luego en qa y después en staging, se detectaron y solucionaron errores en etapas tempranas, evitando que fallos llegaran a usuarios finales.

- El flujo de ambientes permitió:
- Pruebas funcionales en qa.
- Pruebas de comportamiento casi real en staging.
- Validaciones finales antes del despliegue.

Esto disminuyó significativamente los riesgos de fallos en la rama main.

3. Mayor control en los cambios

Cada HU avanzó por ambientes mediante ramas específicas, lo que permitió:

- Rastrear fácilmente qué cambios pertenecen a qué funcionalidad.
- Separar trabajo en paralelo sin interferencias.
- Evitar que una HU afecte accidentalmente el trabajo de otra.
- Además, el uso de cherry-pick permitió trasladar cambios puntuales sin hacer merges no deseados.

4. Mejor organización del equipo y del flujo de trabajo

El manejo de ambientes ayudó al equipo a trabajar de manera más sincronizada y estructurada:

- Se establecieron roles y responsabilidades en cada etapa.
- Se normalizó el proceso de creación y revisión de MR.
- Se adoptaron prácticas colaborativas más profesionales.

Esto fomentó un ambiente de trabajo ordenado que se acerca a los estándares de la industria.

5. Incremento en la trazabilidad y auditoría del proyecto

Gracias al sistema de ambientes:

- Cada fase del desarrollo quedó documentada.
- Fue posible revisar fácilmente el historial de cambios y responsables.
- Se generaron ramas por HU que permiten auditorías y retrocesos sin riesgo.

Esto contribuye a la escalabilidad futura del proyecto.

6. Impacto educativo y profesional

Aunque el backend y móvil tuvieron limitaciones, el aprendizaje adquirido es muy valioso:

- El equipo comprendió las buenas prácticas de GitFlow.
- Se aplicaron técnicas avanzadas como cherry-pick, releases y ramas por ambiente.
- Se desarrolló criterio profesional para futuros proyectos empresariales.

15. Conclusiones

- El manejo de ambientes permitió un flujo organizado y controlado.
- El **frontend** fue el repositorio con implementación correcta y completa.
- El **backend y móvil** presentaron fallos, pero lograron mantener un flujo funcional.
- Este esquema de ambientes facilita revisiones, pruebas y despliegues seguros.
- Es una práctica recomendada para futuros desarrollos del Sistema Autogestión SENA.